

模块 7：工程实战 — 两大工具深度使用

模块概述

核心问题：如何在真实工程中高效使用 AI 编程 Agent？

前面六个模块我们拆解了 AI 编程 Agent 的原理与策略。现在进入实战——把理论落地到日常开发中。

本模块聚焦两款成熟的终端 AI 编程工具：**Claude Code** 和 **OpenCode**。它们代表了两种典型路线：

维度	Claude Code	OpenCode
模型绑定	Claude 模型（Anthropic 原生）	多模型支持（OpenAI / Anthropic / 本地模型）
架构模式	单 Agent + Agent Teams	单 Agent
代码感知	工具链（Glob/Grep/Read）	LSP 集成 + 工具链
扩展性	MCP Server / 自定义 Agent	自定义工具 / Provider
界面	CLI + IDE 集成	TUI（Terminal UI）

7.1 Claude Code 工程全貌

安装与初始化

Claude Code 是 Anthropic 官方的 CLI 工具，安装方式：

```
# 通过 npm 安装
npm install -g @anthropic-ai/claude-code

# 验证安装
claude --version

# 首次运行，完成认证
```

```
claude
```

```
# 在项目中初始化配置
```

```
claude /init
```

`/init` 命令会在项目根目录生成一个 `CLAUDE.md` 文件，这是 Claude Code 的核心配置之一。

CLAUDE.md：项目级指令系统

CLAUDE.md 是 Claude Code 最独特的设计之一。它不是普通的文档——它是**嵌入到 Agent 系统提示中的项目级指令**。每次 Claude Code 启动时，都会读取 CLAUDE.md 并将其内容作为上下文注入。

三级继承体系



三级指令会被合并（merge），优先级从高到低：目录级 > 项目级 > 用户级。这意味着你可以在全局设定通用偏好，然后在每个项目中覆盖特定规则。

CLAUDE.md 应该写什么

一个高效的 CLAUDE.md 模板：

```
# CLAUDE.md
```

```
## 项目概述
```

```
这是一个基于 Next.js 14 的电商平台，使用 TypeScript + Prisma + PostgreSQL。
```

```
## 代码规范
```

- 使用 TypeScript strict mode
- 函数命名使用 camelCase，类型命名使用 PascalCase
- 组件使用函数式组件 + hooks，不使用 class 组件
- 所有 API 路由在 app/api/ 目录下
- 使用 Zod 做运行时类型校验

常用命令

- `pnpm dev` - 启动开发服务器
- `pnpm test` - 运行测试 (vitest)
- `pnpm test:e2e` - 运行端到端测试 (Playwright)
- `pnpm lint` - ESLint 检查
- `pnpm db:migrate` - 运行数据库迁移

架构说明

- app/ - Next.js App Router 页面与路由
- lib/ - 共享工具函数和业务逻辑
- components/ - React 组件 (按功能模块划分)
- prisma/ - 数据库 schema 和迁移文件

提交规范

- 使用 Conventional Commits 格式: feat/fix/chore/docs
- PR 描述必须包含变更摘要和测试说明

关键原则: CLAUDE.md 是给 Agent 看的, 不是给人看的。要写得**明确、可执行**, 避免模糊描述。

settings.json: 权限与行为配置

`settings.json` 控制 Claude Code 的运行时行为, 分为全局和项目级两层:

```
~/.claude/settings.json          ← 全局配置
项目根目录/.claude/settings.json ← 项目级配置
```

典型配置示例:

```
{
  "permissions": {
    "allow": [
      "Bash(npm run test)",
      "Bash(npm run lint)",
      "Bash(git status)",
      "Bash(git diff)",
      "Bash(git log)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force)"
    ]
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit",
```

```

        "hooks": [
            {
                "type": "command",
                "command": "echo 'File being edited: $CLAUDE_FILE_PATH'"
            }
        ]
    },
    ],
    "PostToolUse": [
        {
            "matcher": "Bash",
            "hooks": [
                {
                    "type": "command",
                    "command": "echo 'Command completed'"
                }
            ]
        }
    ]
}

```

完整工具链

Claude Code 内置的工具集直接映射了 Agent 的感知-规划-执行循环：

感知层 (Grounding)

- ├─ Read ─ 读取文件内容
- ├─ Glob ─ 按 pattern 搜索文件路径 (如 `**/*.tsx`)
- ├─ Grep ─ 按正则搜索文件内容 (基于 `ripgrep`)
- ├─ WebFetch ─ 获取并分析网页内容
- └─ WebSearch ─ 搜索网络信息

执行层 (Action)

- ├─ Edit ─ 精确的字符串替换编辑
- ├─ Write ─ 创建或完整覆写文件
- ├─ Bash ─ 执行 shell 命令
- └─ NotebookEdit ─ 编辑 Jupyter notebook

协作层 (Collaboration)

- ├─ Agent ─ 生成子 Agent 处理子任务
- ├─ SendMessage ─ Agent 间通信
- ├─ TaskCreate ─ 创建任务
- ├─ TaskUpdate ─ 更新任务状态
- └─ TaskList ─ 列出所有任务

辅助层

- └─ AskUserQuestion – 向用户提问以获取澄清
- └─ EnterPlanMode – 进入规划模式，先制定方案再执行

常用 Slash 命令

命令	用途
/init	初始化项目 CLAUDE.md
/compact	压缩对话历史，释放上下文空间
/model	切换模型（如 opus → sonnet 节省成本）
/help	查看帮助信息
/clear	清空当前对话
/cost	查看当前会话的 token 消耗和费用

IDE 集成

Claude Code 深度集成到 VS Code（官方扩展，支持选中代码直接发送给 Agent、终端面板内嵌交互界面）和 JetBrains（IntelliJ、WebStorm、PyCharm 等，通过插件支持）。

7.2 Agent Teams

核心动机与团队创建

单 Agent 面临上下文窗口限制、串行执行瓶颈和关注点发散三个问题。Agent Teams 通过**分工并行**解决——每个子 Agent 拥有独立上下文，专注单一任务，通过任务系统协调。

```
TeamCreate({
  team_name: "feature-auth",
  description: "实现用户认证功能"
})
→ 生成 ~/.claude/teams/feature-auth/config.json（团队配置）
→ 生成 ~/.claude/tasks/feature-auth/（任务列表）
```

任务管理：TaskCreate / TaskUpdate / TaskList

任务系统是 Agent Teams 的协调中枢：

```
# 创建任务
TaskCreate({
  subject: "实现 JWT 认证中间件",
  description: "在 lib/auth/middleware.ts 中实现 JWT 验证...",
  activeForm: "Implementing JWT middleware"
})

TaskCreate({
  subject: "编写认证 API 路由",
  description: "在 app/api/auth/ 下创建 login/register/logout...",
  activeForm: "Creating auth API routes"
})

TaskCreate({
  subject: "编写认证功能的测试",
  description: "为 JWT 中间件和 API 路由编写单元测试...",
  activeForm: "Writing auth tests"
})
```

任务支持**依赖关系**——通过 `addBlockedBy` 确保执行顺序：

```
# 测试任务依赖于实现任务
TaskUpdate({
  taskId: "3",
  addBlockedBy: ["1", "2"]    ← 测试必须在中间件和 API 实现后才能开始
})
```

生成子 Agent

使用 `Agent` 工具生成具有不同职责的 Teammate：

```
# 生成后端实现 Agent
Agent({
  name: "backend-dev",
  subagent_type: "general-purpose",
  team_name: "feature-auth",
  prompt: "你负责实现后端认证逻辑。请从 TaskList 获取你的任务。",
  run_in_background: true
})

# 生成测试编写 Agent
Agent({
  name: "test-writer",
  subagent_type: "general-purpose",
  team_name: "feature-auth",
  prompt: "你负责编写测试。请从 TaskList 获取你的任务。",
  run_in_background: true
})
```

```
    })

    # 生成代码审查 Agent
    Agent({
      name: "reviewer",
      subagent_type: "general-purpose",
      team_name: "feature-auth",
      prompt: "你负责审查其他 Agent 提交的代码。请从 TaskList 获取你的任务。",
      run_in_background: true
    })
  })
}
```

关键参数：

参数	说明
name	Agent 名称，用于消息通信和任务分配
subagent_type	Agent 类型：general-purpose（全能）、Explore（只读探索）、Plan（规划）
team_name	所属团队名称
run_in_background	是否后台运行（true 则不阻塞主 Agent）
isolation: "worktree"	在 Git worktree 中运行，避免文件冲突

Agent 间通信

通过 SendMessage 工具实现 Agent 间通信，支持三种类型：

```
# 点对点消息
SendMessage({ type: "message", recipient: "backend-dev",
  content: "JWT 中间件需要支持 refresh token", summary: "JWT refresh token req" })

# 广播（慎用，发给所有 Agent）
SendMessage({ type: "broadcast",
  content: "API base URL 已改为 /api/v2", summary: "API URL changed" })

# 关闭 Agent
SendMessage({ type: "shutdown_request", recipient: "test-writer",
  content: "任务完成" })
```

实际 workflows 示例

一个完整的多 Agent 功能开发流程：

```
Team Lead (主 Agent)
|
├── 1. 分析需求，创建团队和任务
|   TeamCreate("feature-search")
|   TaskCreate("实现搜索 API")           → Task #1
|   TaskCreate("实现搜索 UI 组件")       → Task #2
|   TaskCreate("编写搜索功能测试")       → Task #3 (blocked by #1, #2)
|   TaskCreate("更新文档")               → Task #4 (blocked by #3)
|
├── 2. 生成子 Agent 并分配任务
|   Agent("api-dev")           → 分配 Task #1
|   Agent("ui-dev")           → 分配 Task #2
|   (Task #3, #4 暂时 blocked)
|
├── 3. 子 Agent 并行工作
|   api-dev: 读代码 → 实现 API → 标记 Task #1 完成
|   ui-dev:  读代码 → 实现组件 → 标记 Task #2 完成
|
├── 4. Task #3 解除阻塞
|   Agent("tester")           → 分配 Task #3
|   tester: 编写测试 → 运行测试 → 修复失败 → 标记完成
|
├── 5. Task #4 解除阻塞
|   Team Lead 自己完成文档更新
|
└── 6. 清理
    SendMessage(shutdown_request → 所有子 Agent)
    TeamDelete()
```

最佳实践：只读任务用 `Explore Agent`（更低成本）；可能有文件冲突的 Agent 用 `isolation: "worktree"`；任务粒度不宜过细（至少数个文件的改动）；团队规模控制在 3-5 个 Agent。

7.3 OpenCode 工程全貌

安装与基本配置

OpenCode 是一个开源的终端 AI 编码助手，特点是多模型支持和 TUI 界面。


```
# 通过 Go 安装
go install github.com/opencode-ai/opencode@latest

# 或通过 npm
npm install -g opencode-ai

# 在项目目录中启动
cd your-project
opencode
```

多模型配置

OpenCode 的核心优势是灵活的多模型配置。配置文件位于项目根目录的

`opencode.json`：

```
{
  "providers": {
    "anthropic": {
      "apiKey": "${ANTHROPIC_API_KEY}",
      "disabled": false
    },
    "openai": {
      "apiKey": "${OPENAI_API_KEY}",
      "disabled": false
    },
    "ollama": {
      "baseURL": "http://localhost:11434",
      "disabled": false
    }
  },
  "models": {
    "claude-sonnet": {
      "provider": "anthropic",
      "model": "claude-sonnet-4-6",
      "maxTokens": 16384
    },
    "gpt-4o": {
      "provider": "openai",
      "model": "gpt-4o",
      "maxTokens": 16384
    },
    "deepseek-local": {
      "provider": "ollama",
      "model": "deepseek-coder-v2:latest",
      "maxTokens": 8192
    }
  }
}
```

```
}  
}
```

多模型策略：复杂架构设计用 Opus/GPT-4o（深度推理）、日常编码用 Sonnet/GPT-4o-mini（速度与质量平衡）、敏感代码用本地模型（数据不出网）、批量处理用低成本模型。

LSP 集成

OpenCode 的一大差异化特性是 **LSP（Language Server Protocol）集成**。这使得 Agent 不仅能做文本级的代码搜索，还能理解代码的语义结构：

传统 Agent 感知：
Grep("handleAuth") → 找到所有包含 "handleAuth" 的行
❑ 无法区分：定义 vs 引用 vs 注释中的提及

LSP 增强感知：
GoToDefinition("handleAuth") → 精确跳转到函数定义
FindReferences("handleAuth") → 找到所有调用点
GetDiagnostics() → 获取编译错误和警告
❑ 语义级别的代码理解

LSP 支持的能力：

LSP 功能	Agent 收益
Go to Definition	精确定位函数/类/变量定义
Find References	分析变更的影响范围
Diagnostics	实时获取类型错误和语法错误
Code Actions	获取 IDE 级别的修复建议
Hover Information	获取类型签名和文档

Session 管理

OpenCode 支持 session 的持久化与恢复：

```
# 列出历史 session  
opencode sessions list
```

```
# 恢复之前的 session
opencode sessions resume <session-id>
```

TUI 特性

OpenCode 的 TUI 提供了实时状态可视化（可见 Agent 正在执行的工具）、多面板布局（对话/差异/终端并列）、快捷键操作和 Session 快速切换等增强体验。

7.4 真实工作流

场景 1: Bug 修复全流程

一个典型的 Agent 驱动 bug 修复流程：

用户报告：“登录接口偶尔返回 500 错误，日志中有 `TypeError: Cannot read property 'email' of null`”

用户输入：

```
> 修复登录接口的 500 错误。错误日志：
> TypeError: Cannot read property 'email' of null
> 出现在 app/api/auth/login/route.ts
```

Agent 执行过程：

Step 1: 读取相关代码

工具: `Read("app/api/auth/login/route.ts")`

```
// route.ts
export async function POST(req: Request) {
  const body = await req.json()
  const user = await db.user.findUnique({
    where: { email: body.email }
  })
  const isValid = await bcrypt.compare(body.password, user.passwordHash)
  // ^^^^^ 当 user 为
null 时崩溃
  ...
}
```

Step 2: 分析根因

Agent 识别出：当 email 不存在时，user 为 null，
直接访问 `user.passwordHash` 导致 `TypeError`

Step 3: 搜索类似模式

工具: `Grep("findUnique")`

→ 发现其他 3 个文件有类似的未检查 `null` 的模式

Step 4: 修复代码

工具: `Edit("app/api/auth/login/route.ts")`

old_string: |

```
const isValid = await bcrypt.compare(body.password, user.passwordHash)
```

new_string: |

```
if (!user) {
```

```
  return Response.json({ error: "Invalid credentials" }, { status: 401 })
```

```
}
```

```
const isValid = await bcrypt.compare(body.password, user.passwordHash)
```

Step 5: 运行测试

工具: `Bash("pnpm test app/api/auth")`

✓ 15 tests passed

Step 6: 提交

工具: `Bash("git add app/api/auth/login/route.ts")`

工具: `Bash("git commit -m 'fix: handle null user in login route'")`

场景 2: 功能开发全流程

需求: “给用户资料页添加头像上传功能（JPG/PNG，最大 5MB，上传到 S3）”

Agent 执行过程:

Step 1: 进入规划模式 → `EnterPlanMode()`

Agent 探索代码库:

- `Glob("**/profile/**")` → 找到资料页相关文件
- `Read("app/profile/page.tsx")` → 理解现有页面结构
- `Read("lib/db/schema.prisma")` → 确认 `User` 模型结构
- `Grep("S3|upload|aws")` → 查找现有的 S3 配置

输出计划:

1. 更新 Prisma schema, 添加 `avatarUrl` 字段
2. 创建 S3 上传工具函数 `lib/storage/s3.ts`
3. 创建 API 路由 `app/api/user/avatar/route.ts`
4. 在资料页添加上传组件
5. 编写测试

Step 2-6: 用户批准后, Agent 按计划逐步执行

Step 7: 创建 PR

```
Bash("git checkout -b feat/avatar-upload")
Bash("git add -A && git commit -m 'feat: add avatar upload'")
Bash("gh pr create --title 'feat: add avatar upload' --body '...')"
```

场景 3: 代码审查辅助

用户输入: > 审查 PR #142

Agent 执行:

1. Bash("gh pr view 142 --json title,body,files") + Bash("gh pr diff 142")
2. 逐文件审查: 逻辑正确性、错误处理、类型安全、安全漏洞、性能影响
3. 输出结构化审查意见:

PR #142 审查意见

问题

1. lib/api/handler.ts:45 - 未处理 null 返回值, 可能导致运行时错误
2. app/api/data/route.ts:23 - SQL 拼接存在注入风险, 建议参数化查询
3. lib/cache.ts:12 - 缓存未设置过期时间, 可能导致内存泄漏

总体评价

功能实现正确, 但需要修复安全问题后再合并。

7.5 大型代码库策略

核心矛盾

Agent 的上下文窗口 (即使 200K tokens) 无法装下一个数十万行的 Monorepo, 但修复一个 bug 可能需要理解跨越多个 package 的调用链。以下四个策略帮助 Agent 在大型代码库中高效工作。

策略 1: CLAUDE.md 充当架构地图

在大型项目的 CLAUDE.md 中提供架构导航信息:

CLAUDE.md (大型项目版)

架构概览

本项目采用 Monorepo 结构 (pnpm workspace)。

模块依赖关系

packages/shared → 被所有其他 package 引用
packages/api → 依赖 shared, 提供 REST + GraphQL API
packages/web → 依赖 shared + api 的类型定义
packages/mobile → 依赖 shared, 使用 React Native

关键入口点

- API 服务入口: packages/api/src/index.ts
- Web 应用入口: packages/web/app/layout.tsx
- 共享类型定义: packages/shared/src/types/index.ts
- 数据库 Schema: packages/api/prisma/schema.prisma

模块职责速查

目录	职责	负责团队
packages/api/src/routes/	API 路由定义	后端组
packages/api/src/services/	业务逻辑层	后端组
packages/web/app/	页面与路由	前端组
packages/web/components/	UI 组件	前端组
packages/shared/src/utils/	工具函数	平台组

这份信息让 Agent 在开始探索前就知道**去哪里找**——相当于给它一张地图。

策略 2：定向探索（Glob + Grep 组合拳）

不要让 Agent 漫无目的地浏览代码。用精准的搜索缩小范围：

```
# 第一步：找到相关文件
Glob("packages/api/src/**/*auth*")
→ 找到 5 个认证相关文件

# 第二步：在这些文件中搜索关键逻辑
Grep("validateToken", path="packages/api/src/")
→ 精确定位 token 验证逻辑

# 第三步：追踪调用链
Grep("import.*from.*auth", path="packages/")
→ 找到所有引用认证模块的文件
```

核心原则：先缩小范围，再深入阅读。在大型代码库中，Agent 的每一次 Read 都应该是有目的的，而不是“读一读看看”。

策略 3：分层感知（自顶向下）

层次 1：项目结构（Glob + ls）
→ 了解 package 划分和目录结构
→ 成本：极低（只看文件名）

层次 2：模块接口（Read 入口文件和类型定义）

- 了解模块间的 API 契约
- 成本：中等（只读关键文件）

层次 3：具体实现（Read + Grep 定向搜索）

- 深入特定函数的实现细节
- 成本：较高（读大量代码）

层次 4：运行时行为（Bash 执行测试/调试）

- 验证理解是否正确
- 成本：最高（实际执行代码）

最佳实践：从层次 1 开始，逐层深入，而不是上来就读所有文件。每一层都应该为下一层的搜索提供方向。

策略 4：.claudeignore 限制范围

类似 .gitignore，.claudeignore 告诉 Claude Code 忽略特定文件和目录：

```
# .claudeignore

# 构建产物
dist/
build/
.next/
node_modules/

# 生成代码
packages/api/src/generated/
packages/web/src/__generated__/

# 大型数据文件
*.csv
*.json.gz
data/fixtures/

# 不相关的包
packages/legacy-app/
packages/deprecated-*
```

效果：减少搜索空间、降低上下文污染（生成代码不干扰 Agent 理解）、加快 Glob/Grep 速度。

7.6 何时不该用代理

Agent 不是银弹

理解 Agent 的局限性，与理解它的能力同样重要。盲目使用 Agent 可能导致：

- **时间浪费**：Agent 反复试错的时间超过了人工编码的时间
- **质量隐患**：Agent 生成的代码“看起来对”但存在细微问题
- **过度工程**：Agent 倾向于生成更多代码，而不是更简洁的解决方案

Agent 挣扎的任务类型

1. 高度创造性的设计

- "设计一个创新的用户交互模式"
- "为这个产品设计一套独特的视觉系统"

Agent 擅长的是在已知模式中工作。开创性设计需要人类的审美判断和创造力。

2. 性能优化的“最后一英里”

- "把这个函数的执行时间从 50ms 优化到 5ms"

Agent 能做出通用的优化建议（使用缓存、减少循环嵌套等），但深层的性能优化需要对运行时行为的深度理解——CPU cache 命中率、内存分配模式、GC 压力等。

3. 安全关键代码

- "实现加密密钥轮换逻辑"
- "编写支付处理的核心流程"

安全代码中的一个细微错误可能导致灾难性后果。Agent 生成的代码可能通过所有测试，但存在 timing attack、侧信道泄漏等人类安全专家才能发现的问题。

4. 复杂的状态机和并发逻辑

- "实现分布式锁的续租机制"
- "修复这个 race condition"

并发问题往往无法通过测试复现，需要对执行时序的深刻理解。
Agent 擅长处理确定性逻辑，但对非确定性的并发场景缺乏可靠推理能力。

不值得使用 Agent 的场景

有些任务太简单，使用 Agent 的开销（启动时间、API 费用、上下文加载）反而大于直接手写：

不值得用 Agent：

- 修改一个配置常量
- 重命名一个变量（IDE 重构功能更快）
- 添加一行日志输出
- 简单的 typo 修复

值得用 Agent：

- 跨多个文件的重构
- 理解不熟悉的代码库
- 编写重复性的模板代码
- 从 issue 到 PR 的完整流程

“最后 10%” 问题

Agent 能快速完成 90% 的工作（正确结构、通过基本测试、符合规范），但**最后 10% 需要人类介入**——边界条件的微妙处理、代码风格一致性、用户体验细节、业务规则特殊情况。最高效的工作模式是**混合模式**。

推荐的混合 workflow

阶段 1：Agent 驱动（适合 Agent）

- ├— 代码库探索和理解
- ├— 生成初始实现
- ├— 编写测试框架
- └— 生成文档骨架

阶段 2：人机协作（两者互补）

- ├— Agent 生成代码，人类审查
- ├— 人类指出方向，Agent 执行
- ├— Agent 处理重复性修改，人类处理创造性部分
- └— 人类写核心逻辑，Agent 写测试

阶段 3：人类主导（适合人类）

- ├— 最终代码审查
- └— 性能调优

实操环节

练习 1：使用 Claude Code Agent Teams 完成多文件功能开发

目标：使用 Agent Teams 为一个 Express 应用添加用户注册和登录功能。

步骤：

1. 克隆模板项目并启动 Claude Code: `git clone <template-repo-url> && cd agent-teams-lab && claude`

2. 使用自然语言描述任务：

```
> 使用 Agent Teams 实现用户认证功能。需要：
> 1. 用户注册 API (POST /api/register)
> 2. 用户登录 API (POST /api/login)
> 3. JWT token 验证中间件
> 4. 完整的单元测试
> 请创建一个 3 人团队来并行开发。
```

1. 观察并记录：Agent 如何创建团队和分配任务、子 Agent 之间如何协调、任务依赖如何影响执行顺序

2. 运行 `npm test` 和 `curl -X POST http://localhost:3000/api/register -H "Content-Type: application/json" -d '{"email":"test@example.com","password":"secure123"}'` 验证结果

练习 2：配置 OpenCode 多模型，对比同一任务的输出

目标：配置 OpenCode 接入至少 2 个不同的模型 Provider，对同一代码任务对比不同模型的输出。

步骤：

1. 配置 `opencode.json`

```
{
  "providers": {
```

```
    "anthropic": {
      "apiKey": "${ANTHROPIC_API_KEY}"
    },
    "openai": {
      "apiKey": "${OPENAI_API_KEY}"
    }
  },
  "models": {
    "claude-sonnet": {
      "provider": "anthropic",
      "model": "claude-sonnet-4-6",
      "maxTokens": 16384
    },
    "gpt-4o": {
      "provider": "openai",
      "model": "gpt-4o",
      "maxTokens": 16384
    }
  }
}
```

1. 准备测试任务

向两个模型分别提交以下任务：

重构下面这段代码，提取重复逻辑，添加错误处理：
[准备一段有重复逻辑和缺少错误处理的代码]

1. 对比分析

记录并对比：

对比维度	Claude Sonnet	GPT-4o
代码结构		
错误处理策略		
命名风格		
是否过度工程		
执行速度		
首次正确率		

本模块小结

核心要点回顾

Claude Code 工程体系

- └─ CLAUDE.md – 项目级 Agent 指令（三级继承）
- └─ settings.json – 权限与 Hooks 配置
- └─ 完整工具链 – Read/Edit/Write/Glob/Grep/Bash/Agent
- └─ Agent Teams – 多 Agent 分工协作

OpenCode 差异化优势

- └─ 多模型灵活接入 – 同一工具切换不同模型
- └─ LSP 集成 – 语义级代码感知
- └─ TUI 界面 – 更丰富的终端交互体验
- └─ Session 管理 – 长任务的上下文持久化

大型代码库策略

- └─ CLAUDE.md 架构地图 – 给 Agent 方向感
- └─ 定向搜索 – Glob + Grep 组合拳
- └─ 分层感知 – 自顶向下逐层深入
- └─ .claudeignore – 减少噪音

Agent 使用边界

- └─ 擅长 – 探索、模板代码、跨文件修改、流程自动化
- └─ 挣扎 – 创造性设计、性能优化、安全代码、并发逻辑
- └─ 最佳实践 – 混合模式，Agent 做 90%，人类把控 10%

思考题

- CLAUDE.md 设计：**如果你要为自己当前工作的项目编写 CLAUDE.md，你会包含哪些信息？为什么？试着写出一个初稿，然后评估它是否足够“可执行”。
- Agent Teams 的权衡：**在什么样的任务中，使用 3 个 Agent 并行会比单个 Agent 串行更快？考虑任务分解的开销、文件冲突的风险、以及上下文切换的成本。
- 多模型策略：**假设你的团队同时在开发前端 UI 和后端 API。你会如何配置多模型策略来优化成本和质量？不同的任务类型应该使用什么模型？
- Agent 边界判断：**列举你最近遇到的 5 个编程任务，分别判断它们是否适合使用 Agent。对于“适合”的任务，你会使用什么工作流？对于“不适合”的任务，为什么？

5. **大型代码库挑战：**假设你加入一个拥有 50 万行代码的 Monorepo 项目。你第一天如何利用 Agent 快速建立对项目的整体理解？描述你的具体探索步骤。